

# UniProt ETL: A High-Performance Streaming Engine for Converting UniProtKB XML to Apache Parquet

Filip Rumenovski <sup>1</sup>

<sup>1</sup> Wayne State University, Detroit, MI, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

UniProt ETL is a high-throughput, streaming engine written in Rust that converts UniProtKB XML data dumps into Apache Parquet format. Designed to handle the full scale of UniProtKB—including TrEMBL's 250+ million sequences—the software combines event-driven XML parsing with columnar storage to achieve constant memory usage regardless of input size. On commodity hardware, UniProt ETL processes data at ~24,800 entries/second with ~60× compression while preserving the full biological hierarchy of protein annotations, isoform sequences, and post-translational modification sites.

## Statement of Need

The Universal Protein Resource (UniProt) is the most comprehensive and widely used protein sequence and functional annotation database in the life sciences ([The UniProt Consortium, 2023](#)). UniProtKB/Swiss-Prot alone contains over 570,000 manually curated protein entries, with the complete UniProtKB exceeding 250 million sequences. Researchers in proteomics, structural biology, and machine learning increasingly require programmatic access to this data for tasks ranging from mass spectrometry database searches to training protein language models.

However, working with UniProt's canonical data distribution format—XML dumps exceeding 100 GB uncompressed—presents significant computational challenges. Traditional approaches suffer from fundamental limitations:

1. **Memory constraints:** DOM-based XML parsers (e.g., Python's `xml.etree.ElementTree` or `lxml`) load entire documents into memory, requiring 64+ GB RAM for full datasets and excluding researchers without access to high-memory infrastructure.
2. **Processing time:** Sequential parsing with interpreted languages incurs substantial overhead, and combined with memory-bound operations, can create significant delays in reproducible research pipelines.
3. **Schema flattening:** Ad-hoc conversion scripts often flatten UniProt's hierarchical structure, losing critical relationships between canonical sequences, isoforms, and position-specific annotations.
4. **Isoform handling:** Alternative splice variants require coordinate remapping of features (PTMs, active sites, domains) from canonical to isoform-specific positions—a biologically complex operation that most tools either ignore or implement incorrectly.

Existing solutions include BioPython's SeqIO module ([Cock et al., 2009](#)), which provides convenient parsing but lacks streaming capability and Parquet output; custom pandas-based pipelines, which inherit Python's memory and performance limitations; and vendor-specific database imports, which create lock-in and reproducibility concerns.

40 UniProt ETL addresses these gaps by providing a single, open-source tool that processes  
 41 UniProt XML with constant memory usage (<730 MB peak), preserves biological fidelity  
 42 through rigorous isoform coordinate mapping, and outputs industry-standard Parquet files  
 43 queryable with DuckDB, Polars, Apache Spark, or pandas.

## 44 Software Architecture

### 45 Streaming Pipeline Design

46 UniProt ETL employs a producer-consumer architecture built on three core innovations:

47 **Event-driven XML parsing:** Rather than constructing a DOM tree, the parser uses quick-xml  
 48 (Tarantool, 2024) to process XML as a stream of SAX-like events. Each <entry> element is  
 49 parsed atomically using reusable scratch buffers, then immediately discarded. This bounds  
 50 memory consumption to approximately 1 KB per in-flight entry regardless of input file size.

51 **Bounded-channel I/O decoupling:** The parsing thread (producer) sends RecordBatch structures  
 52 through a bounded crossbeam channel to a dedicated writer thread (consumer). This design  
 53 prevents blocking on Parquet I/O operations while providing natural backpressure to avoid  
 54 memory exhaustion.

55 **Columnar output with nested schemas:** The Arrow/Parquet output preserves UniProt's  
 56 hierarchical structure using LIST and STRUCT types. Features, isoforms, PTM sites, and evidence  
 57 codes are stored as nested arrays, enabling efficient columnar queries without denormalization.

### 58 Isoform Coordinate Mapping

59 A key contribution of UniProt ETL is its rigorous handling of alternative isoforms. UniProt  
 60 entries define splice variants via <feature type="splice variant"> elements, each with a  
 61 unique VSP identifier and location span. Different isoforms reference different subsets of these  
 62 edits.

63 The coordinate mapping algorithm operates as follows:

- 64 1. **VSP-ID scoping:** For each isoform, collect only the splice variant features whose  
 65 identifiers appear in that isoform's reference list.
- 66 2. **Edit accumulation:** Sort edits by position and compute cumulative coordinate shifts  
 67 (insertions add positive deltas; deletions add negative deltas).
- 68 3. **Point mapping:** For a canonical position  $p$  and sorted edit list  $E = [(b_i, e_i, \delta_i)]$ :

$$\text{map}(p) = p + \sum_{i: e_i < p} \delta_i$$

- 69 4. **Ambiguity handling:** Positions falling within length-changing indels (where  $b_i \leq p \leq e_i$   
 70 and  $\delta_i \neq 0$ ) are rejected as unresolvable, preventing biologically meaningless "snapped"  
 71 coordinates.

72 Validation on Swiss-Prot Human data demonstrates 67.6% PTM mapping success (113,466 of  
 73 167,925 sites). The remaining failures are biologically correct rejections: genuine sequence  
 74 variants at the mapped position (24,198), truncated isoforms where the PTM site falls beyond  
 75 the C-terminus (19,652), deletions that physically remove the site (8,275), and ambiguous  
 76 indel interiors (2,148). This biological ceiling represents the maximum achievable fidelity, not  
 77 algorithmic limitations (Rumenovski, 2026).

## Performance

The streaming architecture enables processing of arbitrarily large datasets with bounded memory. Benchmarks on commodity hardware (Apple M4, 16 GB RAM) using Swiss-Prot as a representative workload demonstrate:

| Metric            | Value                  |
|-------------------|------------------------|
| Dataset           | Swiss-Prot (benchmark) |
| Entries processed | 590,000                |
| Throughput        | ~24,800 entries/second |
| Total time        | 23.7 seconds           |
| Peak memory       | ~730 MB                |
| Compression ratio | ~60×                   |

The constant-memory design means these performance characteristics scale to full TrEMBL processing (250M+ entries) without increased RAM requirements—the same ~730 MB footprint handles datasets of any size.

## Research Applications

UniProt ETL enables research workflows at the full UniProtKB scale on commodity hardware:

- **Proteomics database generation:** Rapid construction of organism-specific or PTM-filtered sequence databases for mass spectrometry search engines.
- **Machine learning data preparation:** Efficient extraction of training sets for protein language models (e.g., ESM-2) with verified isoform-mapped annotations.
- **Comparative genomics:** SQL-like queries across the entire proteome using DuckDB or Polars, enabling hypothesis generation without custom parsing code.
- **Reproducible pipelines:** YAML-based configuration and run reports support auditable, version-controlled data processing.

## Availability

UniProt ETL is available under the MIT license at [https://github.com/filiprumenovski/uniprot\\_etl](https://github.com/filiprumenovski/uniprot_etl). The repository includes comprehensive documentation, architecture decision records, integration tests, and example configurations.

## Acknowledgements

The author thanks the UniProt Consortium for maintaining the foundational protein knowledge base that makes this work possible, and the Rust community for the quick-xml, arrow, and parquet crates that underpin the implementation.

## References

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & Hoon, M. J. L. de. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Rumenovski, F. (2026). *ADR-0008: PTM mapping biological ceiling* [Architecture Decision Record]. UniProt ETL Project.

- <sup>110</sup> Tarantool. (2024). *Quick-xml: Rust high performance XML reader and writer*. <https://github.com/tafia/quick-xml>.  
<sup>111</sup>  
<sup>112</sup> The UniProt Consortium. (2023). UniProt: The universal protein knowledgebase in 2023.  
<sup>113</sup> *Nucleic Acids Research*, 51(D1), D483–D489. <https://doi.org/10.1093/nar/gkac1052>

DRAFT